Network Guard

Full Product Explanation for GitHub & Operators

Author: Pilisi W | Year: 2026 | Repository: github.com/pelishminer2-ui/network-guard

1. GitHub Project Card (Screenshot)

The public repository card summarizes the project at a glance: name, visibility, one-line purpose, and primary language.

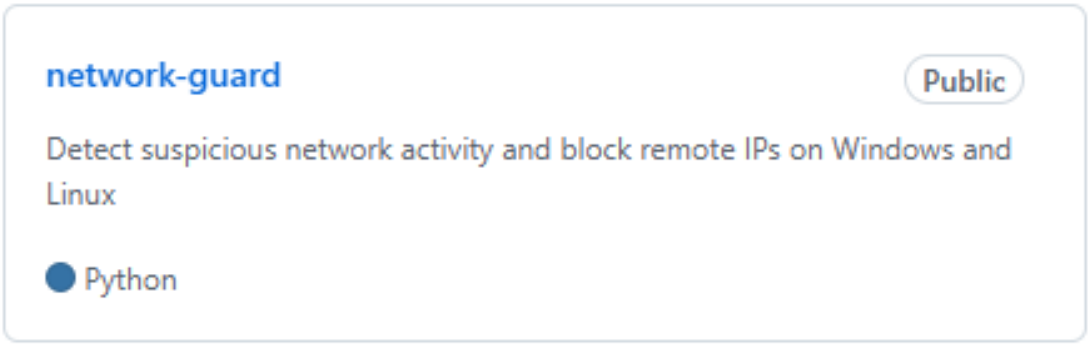

*Figure 1. GitHub repository card for **network-guard** (Public · Python · Detect suspicious network activity and block remote IPs on Windows and Linux).*

2. What This Project Is

Network Guard is a defensive security utility written in Python. It helps an operator (threat analyst) discover suspicious network activity on the local computer and on other devices on the same private LAN, explain what open ports normally mean, optionally watch activity quietly, and then decide whether to **kill** a local process, **block** an IP in the firewall, **allow** it (with memory so unchanged traffic is not asked about again), or **skip**.

It is intended for systems and networks you administer. Blocking performed on one PC protects that PC. Protecting every device on the LAN also requires router-level blocks (a helper file of candidate IPs is written) or installing the companion agent on other machines.

3. Main Components

File	Role
network_guard.py	Core scanner, triage UI, port glossary, allowlist memory, quiet watch, firewall remediation.
run_network_guard.bat	Windows launcher; elevates to Administrator and runs with --lan.
run_network_guard.sh	Linux launcher; uses sudo and --lan.
network_guard_agent.py	Optional agent on other owned PCs; shares screen frames (token-protected) for quiet watch.
run_network_guard_agent.bat	Windows launcher for the agent.
allowlist.txt / blocklist.txt	Trusted and known-bad IPs/CIDRs.

devices.txt	Friendly names (example: 192.168.1.102 → ROKU).
allowlist_state.json	Local fingerprint of allowed traffic; re-prompt only if activity worsens.

4. End-to-End Workflow

4.1 Launch

On Windows, double-click the desktop shortcut or `run_network_guard.bat`. The BAT requests Administrator rights (needed for firewall rules), then runs `network_guard.py --lan` in interactive triage mode. On Linux, `run_network_guard.sh` re-launches with sudo.

4.2 Local socket scan

The script enumerates active TCP/UDP sockets (PowerShell Get-NetTCPConnection / netstat on Windows; ss / netstat on Linux), resolves owning process names, and scores each connection with heuristics:

- Connections to or from ports commonly used by backdoors, reverse shells, proxies, VNC, Tor, or historic Trojans (for example 4444, 31337, 12345).
- Suspicious listening ports on the local machine.
- Exact matches on known abusive tool process names (for example netcat, meterpreter) — exact name match only, to avoid false positives.
- IPs present on the operator blocklist.

4.3 LAN discovery and port probe

With `--lan`, Network Guard auto-detects the home subnet (preferring the network that contains the default gateway, so Hyper-V / WSL virtual adapters are not chosen by mistake). It pings hosts in parallel, merges ARP cache entries, resolves names (devices.txt, DNS, NetBIOS), and probes a focused set of risky/admin ports. Results appear in a LAN inventory table with device names (for example ROKU).

4.4 Port meanings for the analyst

Every flagged or open port is explained in plain English so the person detecting threats understands normal use — for example **3389 = Remote Desktop (RDP)**, **8080 = Alternate HTTP / device admin pages**, **4444 = Common Metasploit/meterpreter listener port**. Explanations appear in the LAN inventory, finding lines, threat reasons, and the Threat View panel.

4.5 Interactive triage

For each finding, Network Guard shows a **Threat View** (process path, command line, window title when available, connections, DNS, device name, MAC, open ports and meanings). The operator then chooses:

- **[U] Quiet watch** — browser live view with no RDP and no window focus steal. Local threats are captured in the background; LAN screens require the Network Guard Agent already installed on that owned PC. Loud RDP mirroring is optional and asked only if RDP is open.
- **[V] Network-only watch** — observe new remote endpoints for ~10 seconds without screen capture.
- **[K] Kill** — terminate the local offending process (not used for remote LAN hosts).
- **[B] Block** — add Windows Firewall or Linux nft/iptables/ufw drop rules for the remote IP (gateway and this PC's own IPs are protected).
- **[KB] Kill and block** — both actions for local threats.
- **[A] Allow** — add IP to allowlist, optional device name, and save a traffic fingerprint so the same behavior is not re-asked unless it gets worse (new suspicious ports/reasons).
- **[S] Skip** / **[Q] Quit** triage.

4.6 Quiet watch details

Quiet mode is the default for **[U]**. It opens the operator's default web browser to a local live page that streams captured frames. It does **not** create a Remote Desktop login session by default, and it does not raise the threat's window to the foreground. That keeps watching from announcing itself via an interactive RDP session. Passwordless RDP, credential brute force, and hidden unauthorized RDP are intentionally not part of this tool.

4.7 Remediation artifacts

- Firewall rules named with a NetworkGuard prefix.
- Appended entries in blocklist.txt or allowlist.txt.
- allowlist_state.json fingerprints for stable allow decisions.
- router_block_candidates.txt listing IPs to paste into router deny lists.
- ui_captures/ snapshots and latest_live.jpg from watch sessions.
- network_guard.log for an audit trail.

5. How Detection Heuristics Work

Detection is signature/heuristic based, not a full IDS. It prioritizes high-signal ports and process names associated with remote access abuse and classic malware listeners, while allowing operators to teach the tool about trusted devices (ROKU, printers, NAS) via allowlist + devices.txt. Allowlisted hosts are remembered; only negative change (worse ports or new threat reasons) brings them back for triage.

6. Platform Support

- **Windows** — netsh advfirewall blocks; taskkill; PowerShell/netstat enumeration; optional mstsc RDP mirror (loud, opt-in).
- **Linux** — nftables / iptables / ufw; kill; ss/netstat.
- **Python 3.7+** recommended; Pillow used for screen capture / PDF assets; stdlib otherwise for core scanning.

7. Typical Operator Commands

Interactive LAN + local scan (default from BAT):

```
python network_guard.py --lan
```

Report only (no firewall changes):

```
python network_guard.py --lan --dry-run
```

Unattended auto-block (use carefully):

```
python network_guard.py --lan --yes
```

LAN devices only:

```
python network_guard.py --lan-only
```

8. Security & Scope Notes (for GitHub readers)

- Run only on networks and devices you are authorized to administer.
- Administrator/root is required for real firewall blocks and process kills.
- False positives are possible; use allowlist and review port meanings before blocking.
- Agent screen sharing requires a shared token file; treat tokens as secrets (agent_token.txt is gitignored).
- This project does not implement credential attacks or stealth unauthorized RDP.

9. Summary

In one sentence: **Network Guard** is a Python defensive tool that scans this PC and your LAN for suspicious sockets and open ports, explains those ports to the analyst, offers quiet browser watching via local capture or an optional agent, and lets you interactively kill, block, allow, or skip each threat — with allowlist memory so stable trusted devices are not nagged again.